

POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY

Faculty of Information Technology

Intelligent System Development

ASSIGNMENT 03

From Data Representation to Deep Learning

"Deep Learning = Function Composition + Representation Learning + Optimization."

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Lưu Anh Dũng
Student ID: B23DCDK036
Semester: I.2025 – 2026
Class: E23CNPM02
Submission: Individual Project

Hanoi, 2026

Contents

1	Executive Summary	2
2	Connection to Lecture 03: Representation Learning and Deep Learning	3
2.1	The central equation	3
2.2	Forward propagation	3
2.3	Loss	4
2.4	Backpropagation	4
2.5	Gradient descent	4
2.6	Weight initialization	4
2.7	What makes this "from scratch"	4
3	Application 1 — Diabetes Prediction	4
3.1	Problem Description	4
3.2	Dataset	5
3.3	Data Understanding and Cleaning	5
3.4	Representation	7
3.5	Classical Machine-Learning Models	8
3.6	From-Scratch Deep Learning Model	8
3.7	Evaluation: All 4 Models	9
3.8	Written Comparison	12
4	Application 2 — House Price Prediction	12
4.1	Problem Description	12
4.2	Dataset	12
4.3	Data Understanding and Cleaning	12
4.4	Representation	15
4.5	Classical Machine-Learning Models	15
4.6	From-Scratch Deep Learning Model	15
4.7	Evaluation: All 4 Models	16
4.8	Written Comparison	19
5	Application 3 — Customer Behaviour (Flipkart Reviews)	20
5.1	Problem Description	20
5.2	Dataset	20
5.3	Data Understanding and Cleaning	20
5.4	Representation	23
5.5	Classical Machine-Learning Models	23
5.6	From-Scratch Deep Learning Model	24
5.7	Evaluation: All 4 Models	24
5.8	Written Comparison	27
6	Cross-Application Comparison	28
7	Discussion: What Changed From Assignment 02	28
8	Conclusion	29
9	Reproducibility	29

1 Executive Summary

This report extends the three intelligent systems from Assignment 02 – diabetes risk screening, house-price regression, and e-commerce review classification – with a deep learning model implemented entirely from scratch using NumPy, with no TensorFlow, PyTorch, or Keras anywhere in the implementation. For each application, four models are trained on an identical train/test split and compared: three classical scikit-learn models and one from-scratch feedforward network whose forward propagation, loss, backpropagation, and gradient-descent update are all derived and coded by hand via the chain rule.

Each dataset was also grown relative to Assignment 02, per the separate verbal requirement that every assignment’s dataset be larger than the one before it, while staying well under a gigabyte in total.

Application	Task	Dataset (rows)
Diabetes	Binary classification	BRFSS 2019+2020 (815,711 raw)
House Price	Regression	USA real estate, 600K sample (2,226,382 raw)
Customer Behaviour	Binary classification	Flipkart reviews (363,261 raw)

Table 1: Overview of the Three Intelligent Systems

1. Diabetes Prediction

- **Dataset:** CDC BRFSS 2019 + 2020 (concatenated), 815,711 raw rows.
- **Problem:** Binary classification – predict whether a survey respondent is diabetic or pre-diabetic.
- **Representation:** Scaled numeric + one-hot nominal + binary-passthrough feature matrix, $X \in \mathbb{R}^{815,711 \times 52}$.
- **4 models:** Logistic Regression, Random Forest, Gradient Boosting, from-scratch DL ($52 \rightarrow 32 \rightarrow 16 \rightarrow 1$).
- **Winner (Accuracy/F1):** Gradient Boosting (Accuracy 0.852, F1 0.286); the from-scratch DL model trails badly on Recall (0.028) due to unweighted BCE on an imbalanced target.

2. House Price Prediction

- **Dataset:** USA Real Estate Listings, 600,000-row sample of 2,226,382 raw listings.
- **Problem:** Regression – predict listing price ($\log(1 + y)$ in USD).
- **Representation:** Scaled numeric + one-hot categorical + frequency-encoded city feature matrix, $X \in \mathbb{R}^{599,522 \times 65}$.
- **4 models:** Linear Regression, Random Forest, Gradient Boosting, from-scratch DL ($65 \rightarrow 64 \rightarrow 32 \rightarrow 1$, linear output).
- **Winner (R^2):** Random Forest (R^2 0.417, RMSE \$794,082); the from-scratch DL model places last (R^2 0.057).

3. E-Commerce Customer Behaviour

- **Dataset:** Flipkart product reviews, 363,261 raw rows (different platform from Assignment 02’s Sephora reviews).
- **Problem:** Binary classification – predict whether a review is a ”recommend” ($\text{Rate} \geq 4$).
- **Representation:** TF-IDF (1–2 gram) over review text + scaled price; classical models use the full 8,695-term vocabulary, the DL model a 2,000-term cap.

- **4 models:** Logistic Regression, Linear SVM, Multinomial Naive Bayes, from-scratch DL ($2,001 \rightarrow 128 \rightarrow 32 \rightarrow 1$).
- **Winner (Accuracy/F1):** Logistic Regression (Accuracy 0.994, F1 0.996); the from-scratch DL model collapses to a majority-class predictor (Accuracy 0.766, Recall 1.0, Precision = Accuracy).

Two findings stand out across all three applications. First, the from-scratch deep learning model never wins its task’s primary metric – it places last (or effectively last) in every one of the three applications. Second, the same underlying cause explains two of the three shortfalls: the from-scratch training loop uses plain, unweighted full-batch gradient descent with no class weighting, no resampling, and a fixed 0.5 threshold, which is vulnerable to exactly the class imbalance present in both classification tasks (84.4%/15.6% for diabetes, 76.5%/23.5% for customer behaviour) in a way the classical models’ training procedures are not. The house-price shortfall has a different, complementary cause: a heavy-tailed regression target colliding with an unbounded linear output layer that lacks an ensemble’s implicit averaging/regularization. All three are genuine, explainable limitations of the from-scratch implementation as built, not implementation bugs – every back-propagation derivation was separately gradient-checked against numerical finite differences before being used to train on real data (see each application’s DL subsection).

2 Connection to Lecture 03: Representation Learning and Deep Learning

This section is written once, at the framework level, so the three per-application DL subsections below do not each have to re-derive the same mathematics – each one instantiates the ideas here with its own concrete widths and numbers, and references this section rather than repeating it.

2.1 The central equation

Lecture 03 frames a deep neural network as a composition of simple functions applied one after another to the input:

$$\hat{y} = (f_L \circ f_{L-1} \circ \cdots \circ f_1)(x)$$

Each f_ℓ is one layer: a linear transformation followed by a nonlinear activation. Stacking L of these layers is what makes the network ”deep.” The lecture’s own closing summary states this compactly: *Deep Learning = Function Composition + Representation Learning + Optimization*. Function composition is the equation above; representation learning is the idea that each layer’s output is a new, learned representation of the input, progressively more useful for the final task than the raw features were; optimization is the process – gradient descent – that adjusts every layer’s weights so the composed function actually produces useful representations rather than arbitrary ones.

2.2 Forward propagation

A forward pass pushes a batch of input rows through every layer in sequence, always in the same two-step order: multiply first, activate second.

$$Z_\ell = A_{\ell-1}W_\ell + b_\ell, \quad A_\ell = g_\ell(Z_\ell)$$

$A_0 = X$ is the input itself, W_ℓ/b_ℓ are that layer’s learned weights and biases, Z_ℓ is the pre-activation, and g_ℓ is the layer’s activation: ReLU for every hidden layer across all three applications, and either sigmoid (binary classification) or no activation at all (regression) at the output layer. Every intermediate value computed during the forward pass is cached, because the backward pass immediately after it needs those exact values to compute gradients.

2.3 Loss

The loss is the single number that measures how wrong the network's current predictions are. Binary classification uses binary cross-entropy, $-\text{mean}(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$, which reduces to $-\log(\text{the probability assigned to the correct class})$ per row; regression uses mean squared error, $\text{mean}((\hat{y} - y)^2)$, which penalizes large misses disproportionately more than small ones because the error is squared before averaging.

2.4 Backpropagation

Backpropagation answers one question per layer, working backward from the output: how much is this layer to blame for the final error, and in which direction should its weights move to reduce it? This is the chain rule applied by hand, one layer at a time. For the output layer, the error signal is $dZ_L = \hat{y} - y$ (classification, sigmoid+BCE) or $dZ_L = 2(\hat{y} - y)/n$ (regression, linear+MSE) – different formulas, because classification's simple form is a known algebraic cancellation between sigmoid's derivative and BCE's derivative that does not occur for a linear output under MSE. Every earlier layer then follows the same repeating pattern: route the error backward through the next layer's weights ($dH_\ell = dZ_{\ell+1} W_{\ell+1}^\top$), gate it by the ReLU derivative ($dZ_\ell = dH_\ell \odot \mathbb{I}[Z_\ell > 0]$, zeroing the gradient for any neuron that was inactive during the forward pass), then compute that layer's parameter gradients ($dW_\ell = A_{\ell-1}^\top dZ_\ell$, $db_\ell = \sum dZ_\ell$).

2.5 Gradient descent

"Learning" is repeating four steps every epoch: **forward** (compute predictions) $\rightarrow$ **loss** (measure how wrong they are) $\rightarrow$ **backward** (compute, via the chain rule above, how much each weight is to blame) $\rightarrow$ **update** (nudge every weight a small step opposite its gradient, $W \leftarrow W - \eta dW$, since the gradient points toward higher loss). Every network in this assignment uses full-batch gradient descent – one forward/backward pass over the entire training set per epoch, no mini-batching – the simplest variant, and the one explicitly described in the lecture material.

2.6 Weight initialization

Every weight matrix is drawn from $\mathcal{N}(0, 1)$ and scaled by $\sqrt{2/\text{fan_in}}$ (He initialization), biases start at zero. Random initialization breaks the symmetry that would otherwise make every neuron in a layer compute the identical thing; the $\sqrt{2/\text{fan_in}}$ scale keeps each layer's output variance roughly stable rather than exploding or vanishing as it passes through multiple layers of ReLU units.

2.7 What makes this "from scratch"

Every one of the six per-layer gradients ($dW_1, db_1, dW_2, db_2, dW_3, db_3$) in every application's network is derived symbolically via the chain rule and implemented as plain NumPy array operations – no autograd engine, no framework. Each application's DL section below states that its specific backward-pass implementation was separately verified against a numerical finite-difference gradient check on synthetic data before being used to train on real data, confirming the hand-derived formulas are correct independent of whether the resulting model performs well.

3 Application 1 — Diabetes Prediction

3.1 Problem Description

Binary classification: predict whether a BRFSS survey respondent is diabetic or pre-diabetic from self-reported lifestyle and biometric indicators, extending Assignment 02's diabetes screening task onto a larger, two-year BRFSS sample.

$$f : \mathbb{R}^{52} \rightarrow \{0, 1\}$$

3.2 Dataset

- **Source:** CDC BRFSS, Kaggle dataset `spandanjit2005/brfss-diabetes-indicator-dataset`.
- **Files used:** `DATASET_2019.csv` + `DATASET_2020.csv`, concatenated with a `year` indicator.
- **Rows:** 815,711 raw $\rightarrow$ 652,568 train / 163,143 test (80/20 stratified split; no rows dropped during cleaning, since all 33 raw columns are retained or encoded rather than filtered out).
- **Growth vs. Assignment 02:** $3.2\times$ the prior 253,680-row BRFSS 2015 sample.
- **Target:** `diabetes` (raw 3-class text: Non-diabetic / Pre-diabetic / Diabetic) collapsed to `Diabetes_binary` $\in \{0, 1\}$ (0 = confirmed non-diabetic, 1 = pre-diabetic or diabetic).

3.3 Data Understanding and Cleaning

The raw target is 84.4% Non-diabetic, 13.8% Diabetic, 2.2% Pre-diabetic; collapsing the latter two gives an 84.4%/15.6% binary split (Figure 1). Seven columns were dropped: five raw-text duplicates of an already-numeric ordinal column (e.g. `general_health_00` duplicating `gen_health_00`), and two columns more than 48% missing across all 815,711 rows. Two $\sim 50\%$ -missing but clinically important columns (`high_bp_00`, `high_cholesterol_00`) were kept rather than dropped, each paired with an explicit `_missing` indicator column recording whether the value was present at all, before median/mode imputation (fit on the training split only) filled the gap. `year` was tested rather than assumed useful: the positive-diagnosis rate is 15.97% in 2019 vs. 15.20% in 2020 (Figure 4) – a modest but real ~ 0.8 -point gap – so it was kept as a single `is_2020` binary flag.

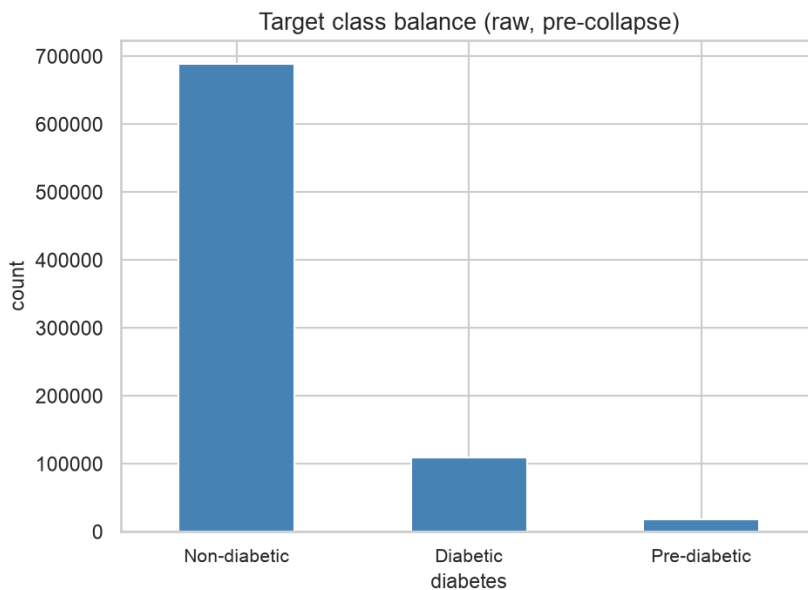


Figure 1: Target class balance, raw 3-class (before collapsing to binary)

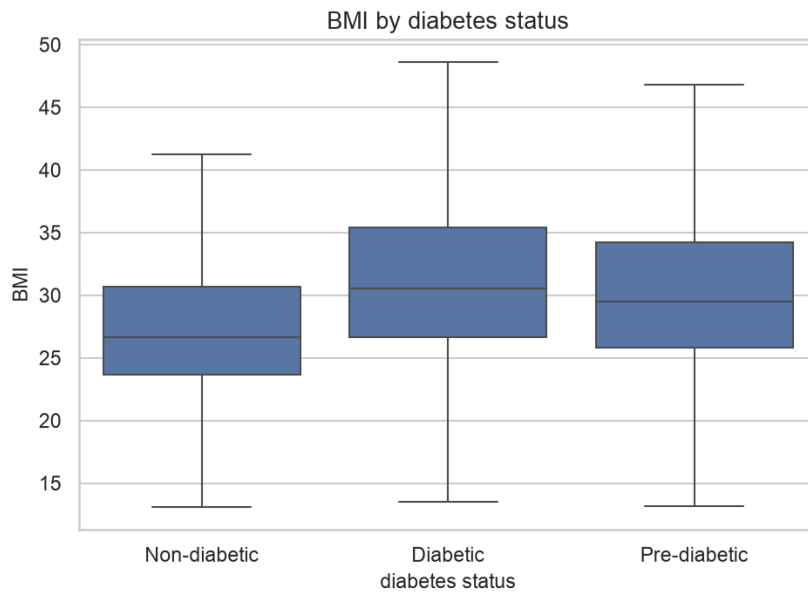


Figure 2: BMI by diabetes status. Mean BMI: 27.7 (non-diabetic) vs. 31.6 (diabetic) – the clinically expected separation.

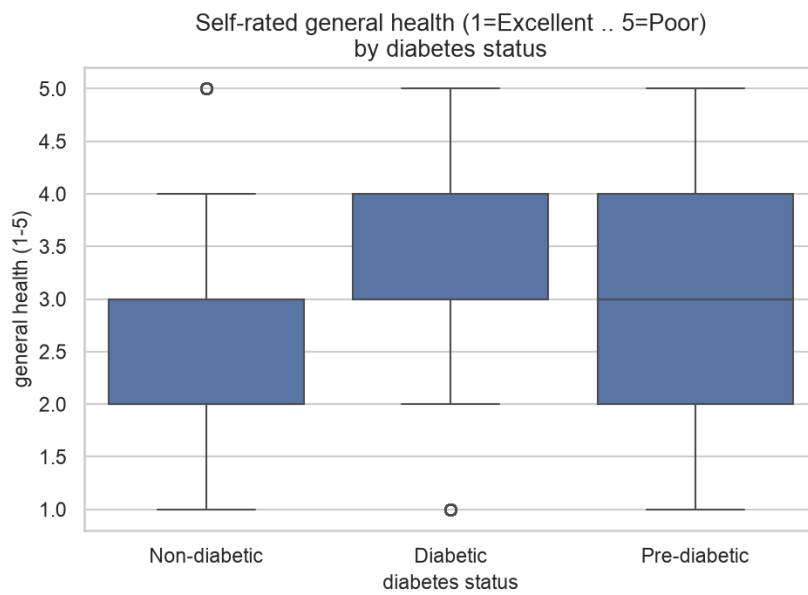


Figure 3: Self-rated general health (1=Excellent..5=Poor) by diabetes status. Mean: 2.40 (non-diabetic) vs. 3.22 (diabetic) – diabetics report meaningfully worse general health.

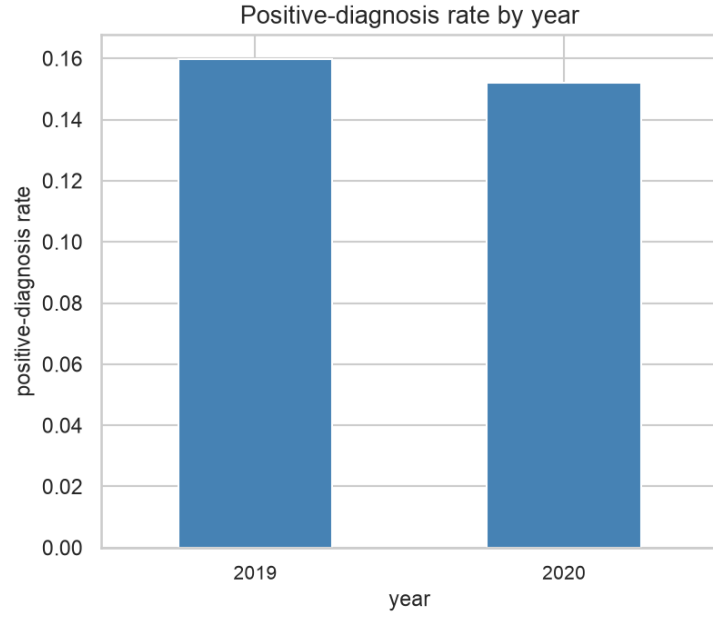


Figure 4: Positive-diagnosis rate by survey year (2019 vs. 2020)



Figure 5: Correlation heatmap: numeric features + target. `gen_health_00` and `bmi_00` show the strongest correlation with `Diabetes_binary` among the numeric columns available at this stage; no pair of input features exceeds $|r| > 0.7$, so no column is dropped on multicollinearity grounds.

3.4 Representation

The final feature matrix combines three blocks, concatenated column-wise after a train/test split computed *before* any fitting step:

- **Scaled numeric** (`StandardScaler`, fit on train only): weight, height, BMI, physical/mental health days, age, education, income, general health, last-checkup recency, plus the two $\sim 50\%$ -

missing clinical columns after imputation – 12 columns.

- **Binary passthrough** (mode-imputed on train, unscaled): sex and 7 boolean survey fields, plus `is_2020` and the two missing-value indicator flags – 11 columns.
- **One-hot nominal** (categories fit on train only, unseen test categories ignored): race, marital status, employment status, personal-doctor status – 29 columns.

Resulting shape: $X \in \mathbb{R}^{815,711 \times 52}$.

3.5 Classical Machine-Learning Models

Model	Why
Logistic Regression	Same functional form as the DL model’s output layer alone – a useful baseline for what depth adds.
Random Forest	Model deployed in Assignment 02 for this app – kept for continuity.
Gradient Boosting	Sequential residual-correction; structurally different from Random Forest’s parallel averaging.

Table 2: Diabetes: 3 classical models and selection rationale

3.6 From-Scratch Deep Learning Model

Architecture: $52 \rightarrow 32 \rightarrow 16 \rightarrow 1$, ReLU hidden layers, sigmoid output, binary cross-entropy loss. $32 \rightarrow 16$ follows the lecture tutorial’s $8 \rightarrow 16 \rightarrow 8 \rightarrow 1$ pattern scaled up for this wider 52-feature input (first hidden layer $\approx$ input width, second layer half that).

Forward propagation: layer 1 linearly recombines the 52 scaled/encoded survey features and applies ReLU, introducing the nonlinearity a purely linear stack could never have (composing linear functions collapses to one linear function, however many layers are stacked); layer 2 recombines layer 1’s learned features into higher-level ones – representation learning in miniature, since layer 2 never sees BMI or blood pressure directly, only whatever combinations of them layer 1 found useful; the output layer applies one more linear combination then sigmoid, squashing the result to a $(0, 1)$ diabetes probability.

Backpropagation: $dZ_3 = \hat{y} - y$ is the output error signal (this simple form is sigmoid’s and BCE’s derivatives canceling algebraically, a known property of this exact pairing); layer 2 asks how much it contributed to that error by routing the signal back through W_3 ($dH_2 = dZ_3 W_3^\top$), then the ReLU derivative zeroes the gradient for any neuron that was inactive during the forward pass ($dZ_2 = dH_2 \odot \mathbb{I}[Z_2 > 0]$); the same route-back→gate→gradient pattern repeats once more for layer 1. This implementation was separately gradient-checked against a numerical finite-difference estimate on synthetic data (maximum relative error $\approx 5 \times 10^{-8}$) before training on the real BRFSS data.

Training: 200 epochs, full-batch gradient descent, learning rate 0.1. Loss fell smoothly from 0.679 to 0.353 with no sign of divergence (Figure 6).

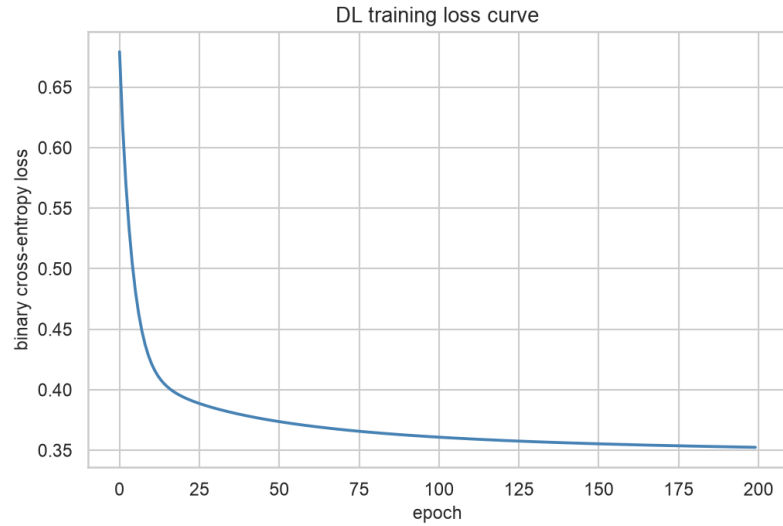


Figure 6: From-scratch DL model: training loss (binary cross-entropy) vs. epoch

3.7 Evaluation: All 4 Models

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.8498	0.5563	0.1814	0.2736
Random Forest	0.8489	0.5494	0.1734	0.2636
Gradient Boosting	0.8516	0.5723	0.1905	0.2859
DL (from scratch)	0.8450	0.5592	0.0284	0.0541

Table 3: Diabetes: 4-model test-set comparison

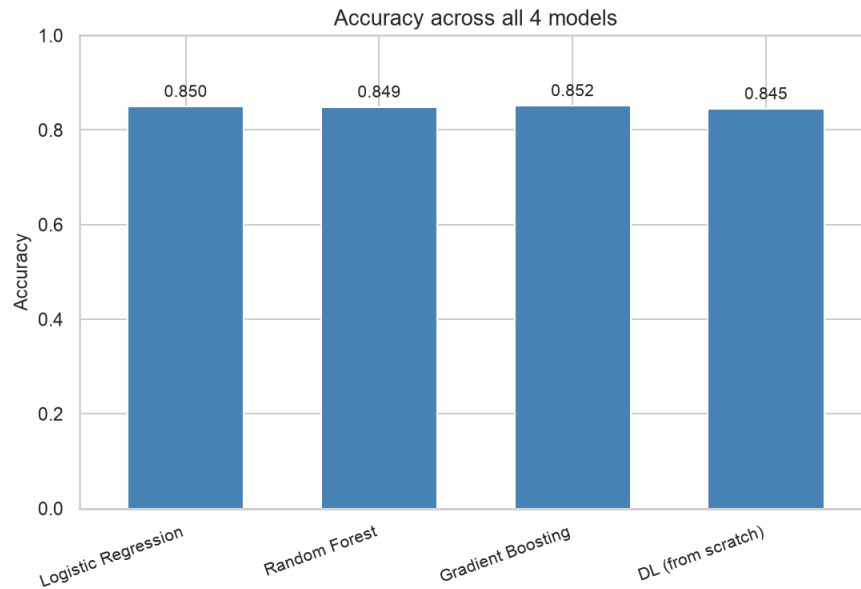


Figure 7: Accuracy across all 4 models

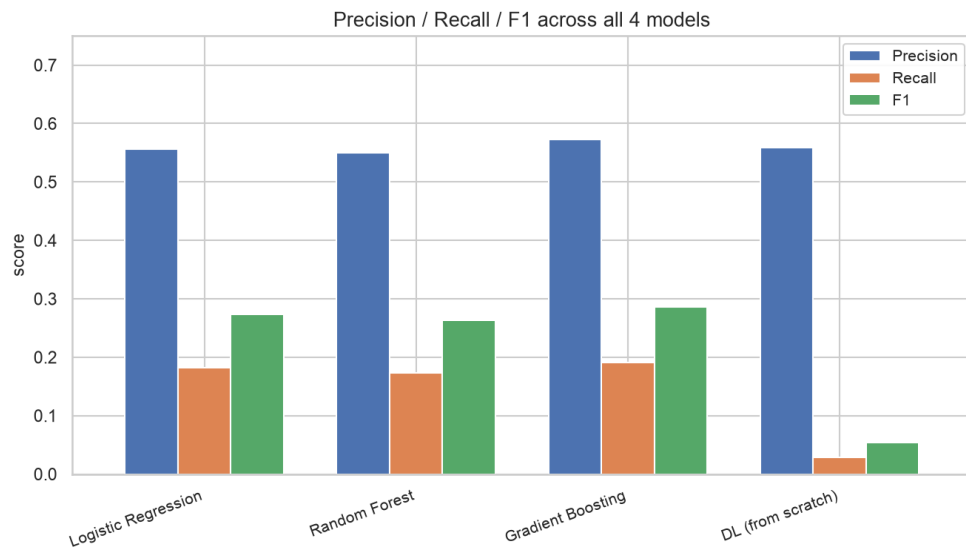


Figure 8: Precision / Recall / F1 across all 4 models

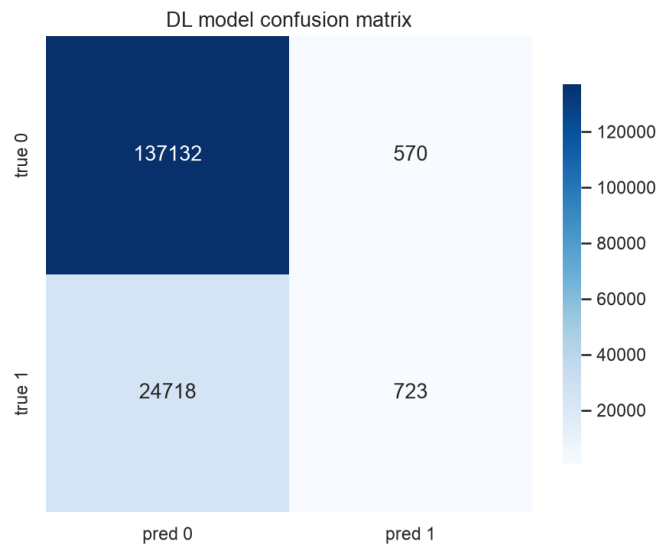


Figure 9: From-scratch DL model: confusion matrix on the test set

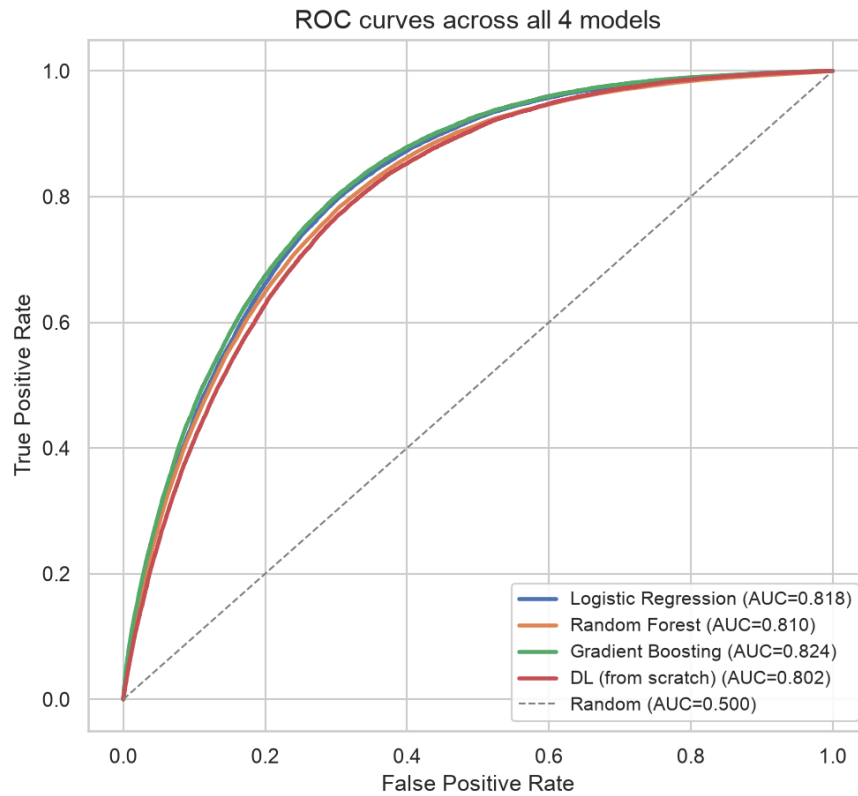


Figure 10: ROC curves across all 4 models. The from-scratch DL model's AUC (0.8025) sits close behind the three classical models (0.8098–0.8239), a much smaller gap than the Accuracy/Recall table above suggests.

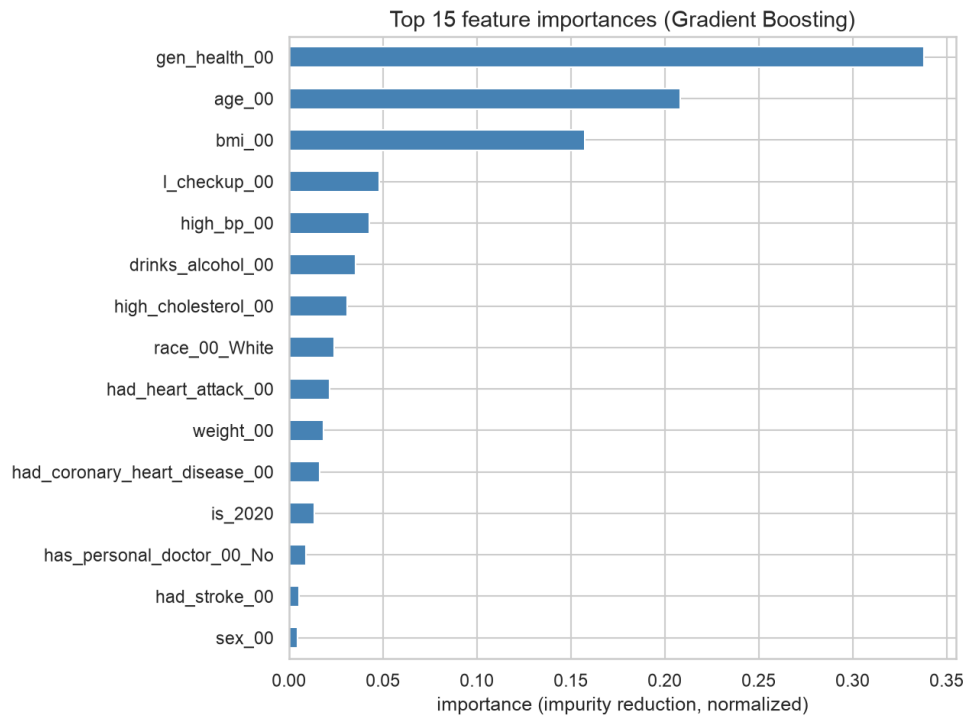


Figure 11: Top 15 feature importances, Gradient Boosting (the winning classical model). **gen_health_00**, **age_00**, and **bmi_00** – the same features the correlation heatmap flagged as most predictive – dominate.

3.8 Written Comparison

On Accuracy, Gradient Boosting wins narrowly (0.8516) ahead of Logistic Regression (0.8498), Random Forest (0.8489), and the from-scratch DL model last (0.8450) – but Accuracy alone is misleading on this 84.4%/15.6%-imbalanced target, since a trivial always-negative classifier already scores $\sim 84.4\%$. Recall is the clinically relevant metric, and here the gap is stark: Gradient Boosting leads at 0.191, Logistic Regression 0.181, Random Forest 0.173, and the from-scratch DL model trails far behind at just 0.028 (723 true positives caught out of 25,441 actual positives in the test set).

The ROC curves in Figure 10 resolve what would otherwise look like a clear failure: the DL model’s AUC (0.8025) sits close behind the three classical models (0.8098–0.8239), not far below them – meaning the network did learn a genuinely useful ranking of who is more/less likely diabetic, it is not confused or undertrained. The Accuracy/Recall gap is specifically a **decision-threshold** problem, not a **ranking** problem: at the fixed 0.5 cutoff, unweighted binary cross-entropy training on an 84.4%-negative target pushed the network’s probability outputs low enough that few positives cross 0.5, even though the relative ordering of those probabilities (what AUC measures) is nearly as good as the tree ensembles’.

In the lecture’s representation-capacity/overfitting/dataset-size framing: this is not an overfitting problem (the loss curve fell smoothly with no divergence) or a representation-capacity problem ($52 \rightarrow 32 \rightarrow 16 \rightarrow 1$ has ample capacity for 52 tabular features, and Figure 11 confirms the features Gradient Boosting relies on most are exactly the ones the correlation heatmap flagged as predictive, so the representation itself is sound) – it is an optimization/class-imbalance problem specific to plain full-batch gradient descent at a fixed threshold. Gradient Boosting’s sequential residual-correction and Random Forest’s per-tree resampling both implicitly re-weight harder (minority-class) examples over training, while plain gradient descent on the full batch does not. This is a genuine, explainable limitation of the from-scratch implementation as built, not a bug – a class-weighted loss term or a tuned decision threshold (Figure 10 shows a lower threshold would recover much of the missing Recall) are natural extensions, left out here to keep the training loop matching the lecture’s exact unweighted BCE formula.

4 Application 2 — House Price Prediction

4.1 Problem Description

Regression: predict a US residential listing’s price from structural and geographic attributes.

$$f : \mathbb{R}^{65} \rightarrow \mathbb{R}^+$$

evaluated via RMSE, MAE, and R^2 in dollar terms after inverting the $\log(1 + y)$ training target.

4.2 Dataset

- **Source:** Kaggle dataset `ahmedshahriarsakib/usa-real-estate-dataset`, `realtor-data.csv`.
- **Rows:** 2,226,382 raw listings, sampled once (seeded, `RANDOM_SEED=42`) to a persisted 600,000-row working set (`realtor-data-sample.csv`), cleaned to 599,522 rows, split 479,617 train / 119,905 test.
- **Growth vs. Assignment 02:** $2.5\times$ the prior 238,924-row Vietnamese real-estate dataset, and a genuinely different (US) market.
- **Target:** price (USD), trained as $\log(1 + \text{price})$ and inverted with `expm1` for evaluation.

4.3 Data Understanding and Cleaning

Raw `price` is heavily right-skewed, with a handful of implausible values including one listing at \$2,147,483,600 – a value suspiciously close to $2^{31} - 1$, a classic 32-bit integer overflow artifact –

and 280 rows at \$0 or negative (Figure 12). Those rows were dropped rather than clipped, since they are not genuine sale prices. `house_size`, `bed`, `bath`, and `acre_lot` showed the same overflow-style corruption (`house_size` up to 1,040,400,400; `bed` up to 473; `bath` up to 830) that drove the raw `house_size`-vs-`price` correlation to an implausible $r \approx 0.0004$ (Figure 13); capping each at its own 99th percentile (rather than dropping rows) raised the correlation to a sensible $r \approx 0.27$ with raw price while preserving 99% of real data. `brokered_by` and `street` were dropped as anonymized, high-cardinality identifiers with no generalizable signal. `prev_sold_date` was engineered into `days_since_prev_sale` (correlation ≈ 0.036 with price – weak but real and free to keep) rather than dropped outright.

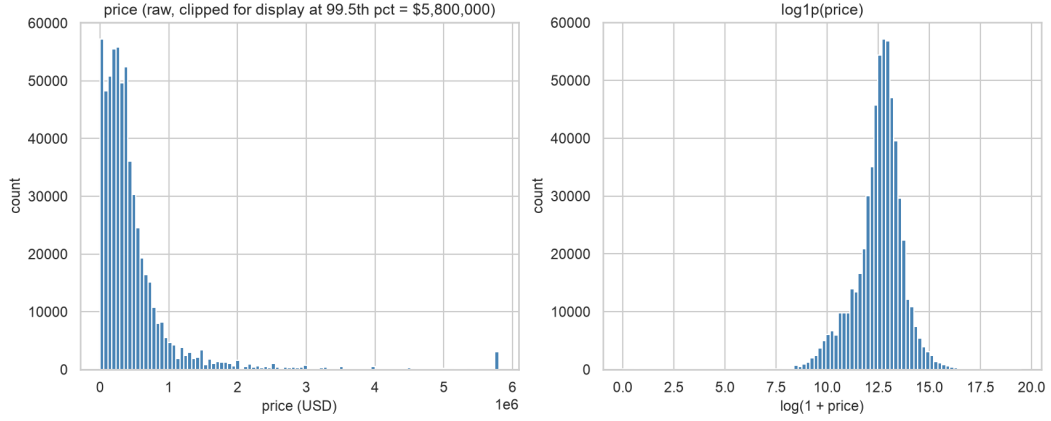


Figure 12: Raw price distribution, clipped for display at the 99.5th percentile (left) vs. $\log(1 + \text{price})$ (right) – the log-transform normalizes the heavy right tail, justifying training in log-space

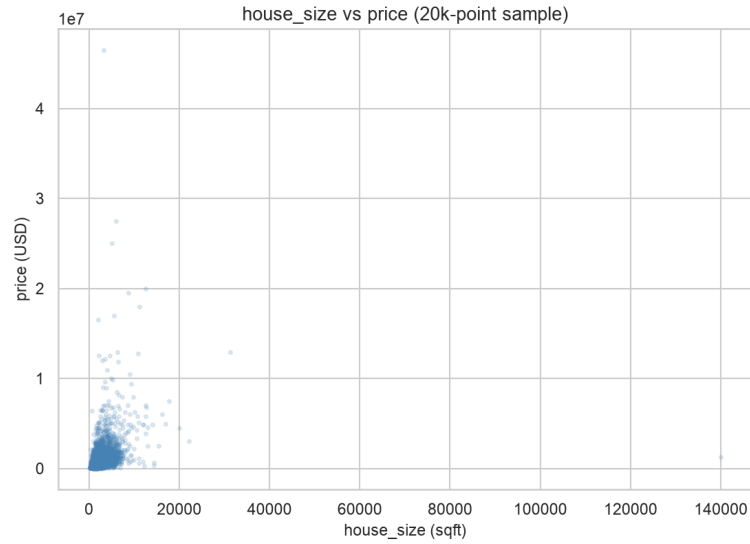


Figure 13: `house_size` vs. `price`, 20K-point sample – reveals the outlier-driven distortion before capping (raw correlation $r \approx 0.0004$)

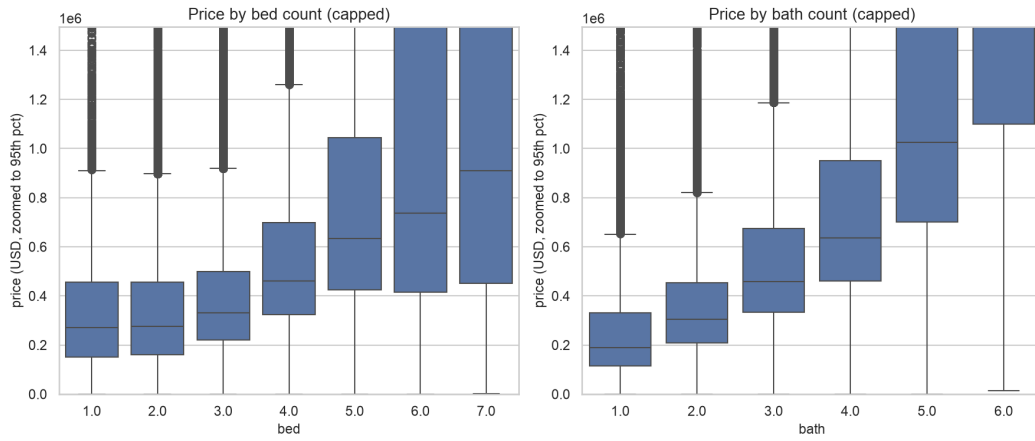


Figure 14: Price by bed count (left) and bath count (right), zoomed to the 95th percentile – both reveal the same outlier-driven distortion in the raw category range before capping

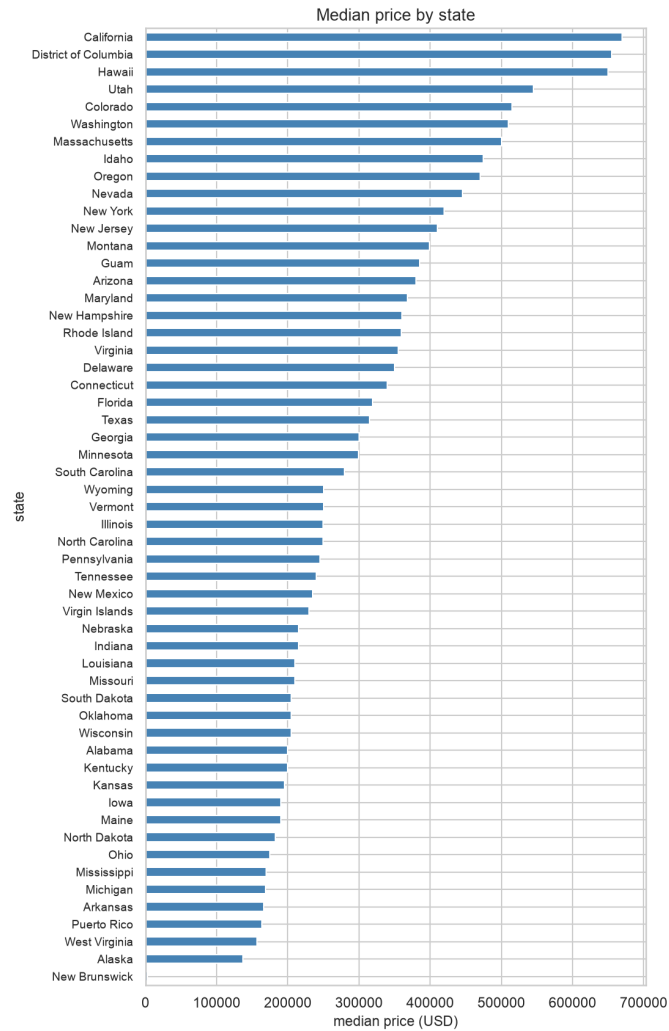


Figure 15: Median price by state – confirms **state** carries real, keep-worthy price signal (California/Hawaii/D.C./Utah highest, Michigan/Arkansas/West Virginia/Alaska lowest)

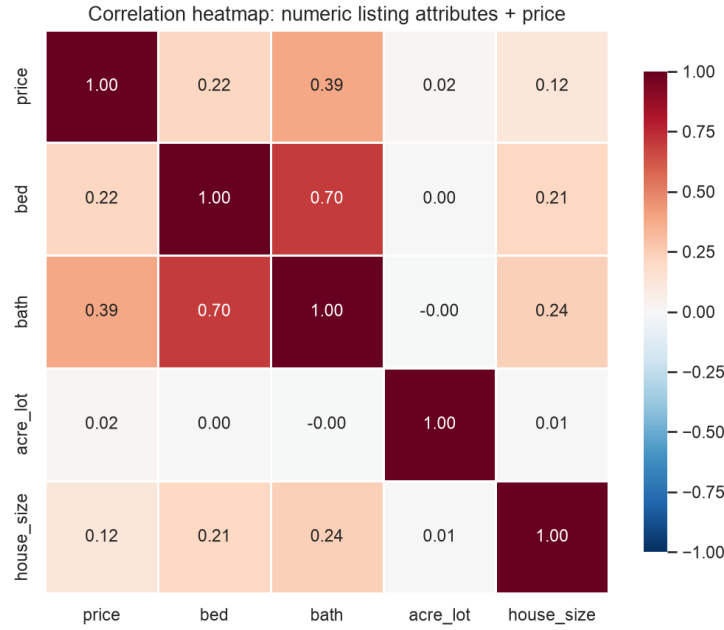


Figure 16: Correlation heatmap: numeric listing attributes + price. **bath** shows the strongest raw correlation with price ($r = 0.39$), followed by **bed** ($r = 0.22$) and **house_size** ($r = 0.12$); **acre_lot** is weakest ($r = 0.02$).

4.4 Representation

- **Scaled numeric** (StandardScaler, fit on train only): capped **bed**, **bath**, **acre_lot**, **house_size**, plus **days_since_prev_sale** – 5 columns.
- **One-hot categorical** (fit on train only): **status** and **state** – 59 columns.
- **Frequency-encoded**: **city** (20,098 distinct values – too high-cardinality for one-hot, but the state-level price spread showed genuine sub-state location signal was worth keeping) – 1 column. **zip_code** was dropped for the same cardinality reason without the same expected payoff.

Resulting shape: $X \in \mathbb{R}^{599,522 \times 65}$.

4.5 Classical Machine-Learning Models

Model	Why
Linear Regression	Simplest baseline every other model is implicitly measured against.
Random Forest	Model deployed in Assignment 02 for this app – kept for continuity (max_depth=20 , found necessary in practice: unbounded depth on 480K rows produced multi-hundred-thousand-node trees and an 8GB+ model file).
Gradient Boosting	Sequential residual-correction, structurally different from Random Forest.

Table 4: House Price: 3 classical models and selection rationale

4.6 From-Scratch Deep Learning Model

Architecture: $65 \rightarrow 64 \rightarrow 32 \rightarrow 1$, ReLU hidden layers, **linear** output (no activation – regression, not classification), MSE loss. Wider hidden layers than diabetes’s despite the smaller input, since log-price’s relationship to bed/bath/size/ location is plausibly more nonlinear, giving the network more room to compose interactions before collapsing to one output.

Forward propagation: identical two-layer ReLU structure to diabetes’s network, but the output layer is deliberately **linear** ($\hat{y} = Z_3$, no sigmoid) – the one required departure from the classification networks’ math, since squashing to $(0, 1)$ would cap predicted log-price at 1, which is wrong for a target that must range freely.

Backpropagation: the one place the math must differ, not just be copied: $dZ_3 = 2(\hat{y} - y)/n$ is MSE’s gradient with respect to a linear output taken directly, with no sigmoid derivative to cancel against (unlike the classification networks’ simpler-looking $\hat{y} - y$ form). Every layer after that follows the identical route-back→ReLU-gate→gradient pattern used in the diabetes network. Separately gradient-checked against a numerical finite-difference estimate (maximum relative error $\approx 4 \times 10^{-9}$) before training on real data.

Training: 200 epochs, full-batch gradient descent, learning rate 0.05 (lower than diabetes’s 0.1, chosen for the wider 64/32 hidden layers’ larger accumulated gradient magnitudes). Loss fell from 152.14 to 1.04, settling by roughly epoch 60 (Figure 17).

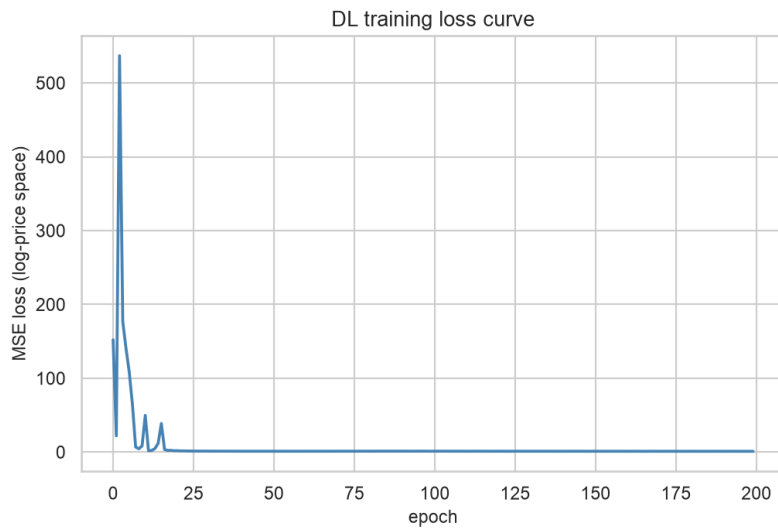


Figure 17: From-scratch DL model: training loss (MSE, log-price space) vs. epoch

4.7 Evaluation: All 4 Models

Model	RMSE (USD)	MAE (USD)	R^2
Linear Regression	894,828	271,849	0.2599
Random Forest	794,082	207,440	0.4172
Gradient Boosting	900,739	244,026	0.2501
DL (from scratch)	1,010,214	328,971	0.0567

Table 5: House Price: 4-model test-set comparison (dollar-space, after expm1 inversion)

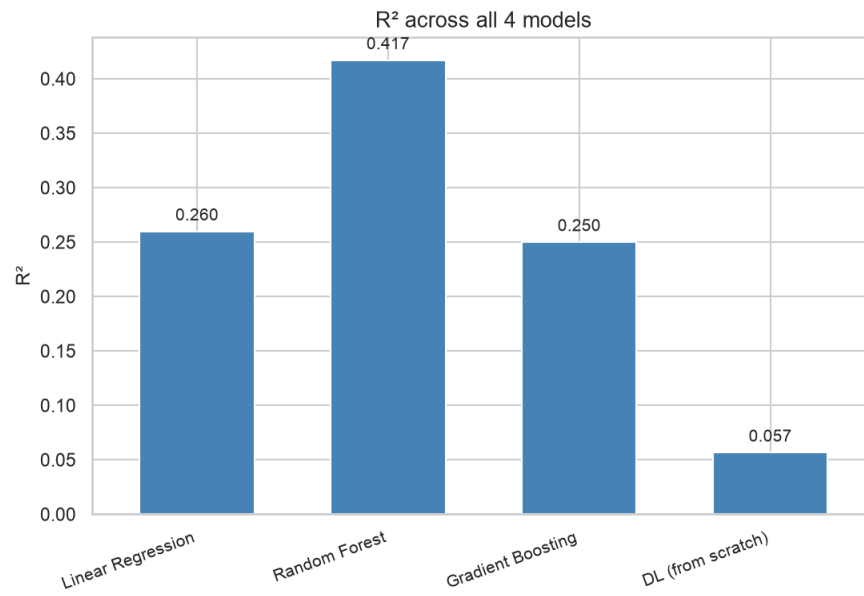


Figure 18: R^2 across all 4 models

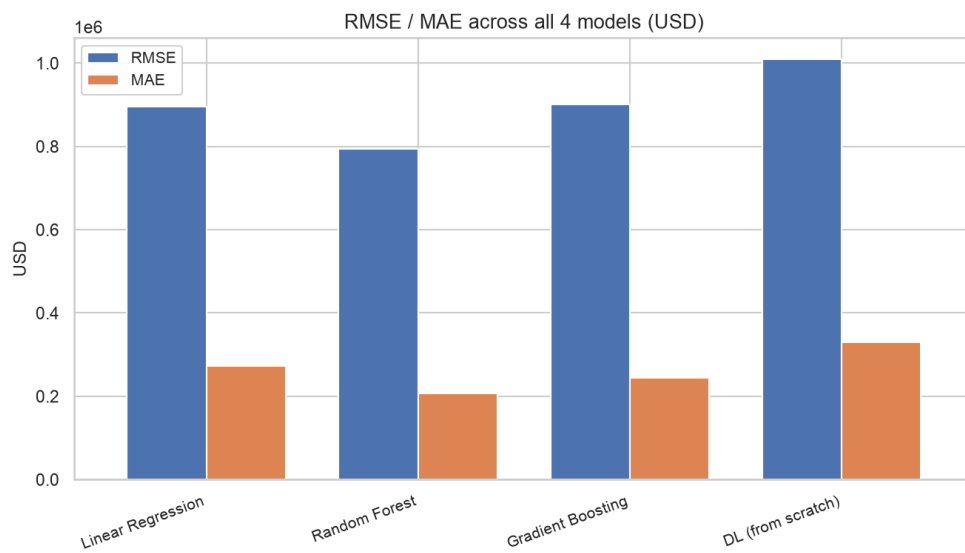


Figure 19: RMSE / MAE in USD across all 4 models

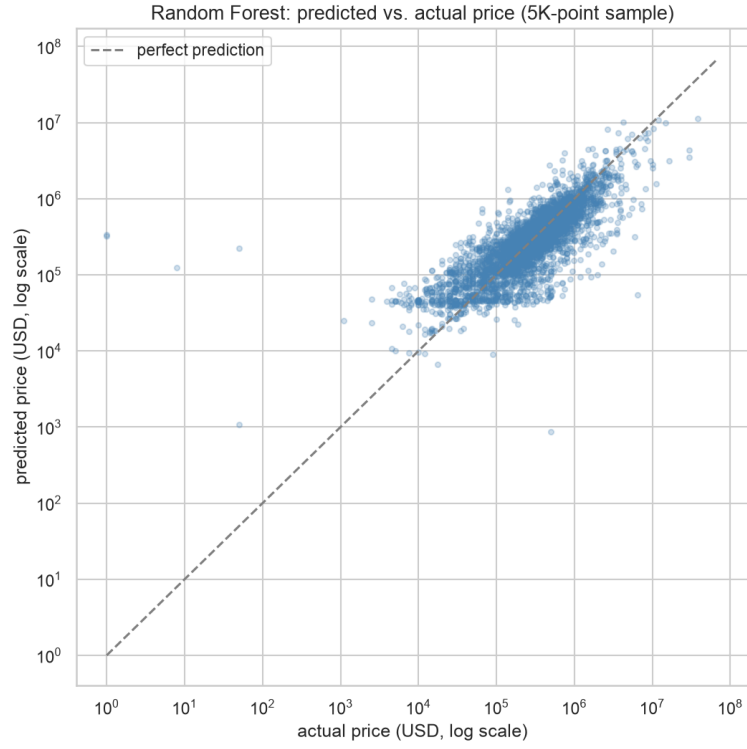


Figure 20: Random Forest: predicted vs. actual price, log-log scale (5K-point test-set sample). Points track the perfect-prediction diagonal reasonably well through the middle of the price range, with spread opening up at both extremes – exactly where a heavy-tailed target is hardest to predict.

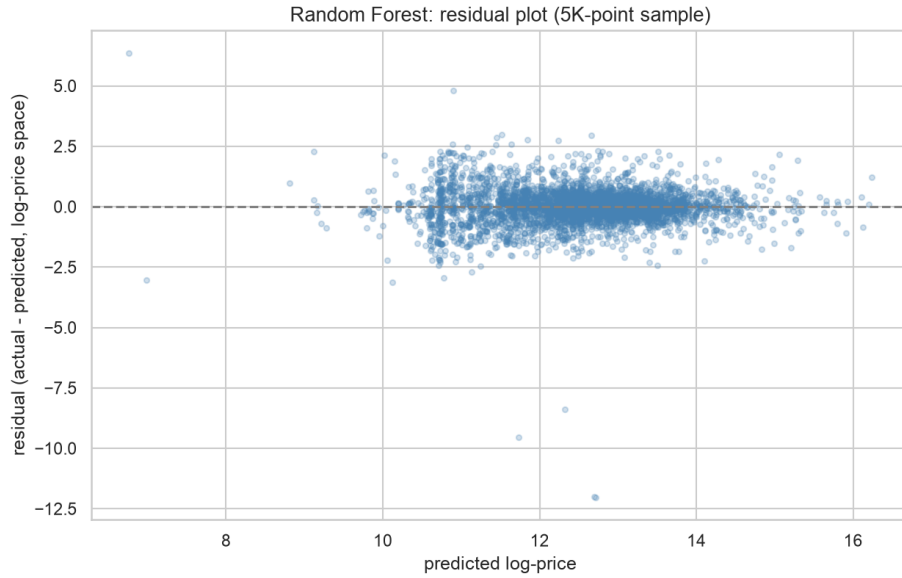


Figure 21: Random Forest: residual plot in log-price space (5K-point test-set sample). Residuals center near zero across most of the predicted range with a mild fan-out at the extremes – a moderately healthy pattern, not a strong funnel or curve.

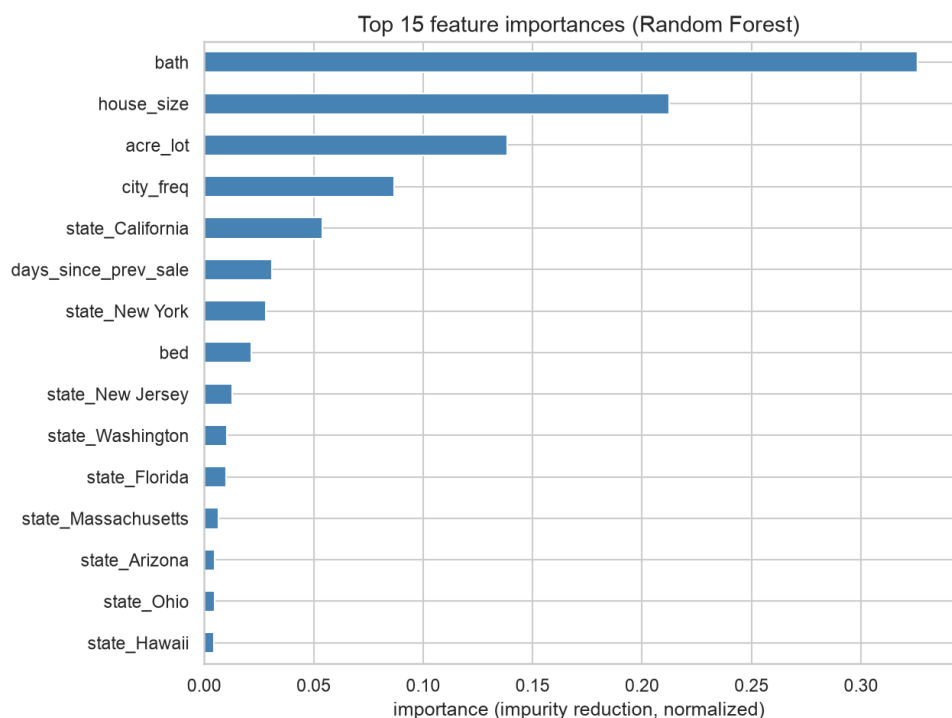


Figure 22: Top 15 feature importances, Random Forest. **bath** dominates (0.326), followed by **house_size** (0.212) and **acre_lot** (0.138) – notably, **acre_lot** ranks far higher here than its weak raw correlation with price suggested, indicating the ensemble captures a nonlinear or interaction effect a simple Pearson correlation cannot.

4.8 Written Comparison

Random Forest wins clearly on every metric – R^2 0.417, RMSE \$794,082, MAE \$207,440 – well ahead of Linear Regression (R^2 0.260), Gradient Boosting (R^2 0.250, sklearn defaults, untuned), and the from-scratch DL model, which places last (R^2 0.057, RMSE \$1,010,214, MAE \$328,971).

Figure 20 shows Random Forest’s predictions tracking the perfect-prediction diagonal well through the bulk of the price range, with the expected spread at the extremes; Figure 21 shows no strong funnel or curvature in the residuals, consistent with a moderately-fit, not badly misspecified, model on a genuinely hard target. Figure 22 adds a further nuance: Random Forest relies most on **bath**, but weights **acre_lot** far more heavily than its raw correlation with price would suggest ($r = 0.02$ vs. an importance of 0.138) – direct evidence that the ensemble is capturing a nonlinear or location-interaction effect (lot size mattering differently by state/city) that the correlation heatmap in Figure 16 alone could not reveal.

In the lecture’s framing: the from-scratch network has comparable raw capacity to the tree ensembles, and its training loss fell smoothly with no divergence, so this is not primarily an optimization failure. The real driver is the target’s shape colliding with this model family’s structure: **price** is extremely heavy-tailed even after the $\log(1 + y)$ transform (log-price ranges from 0.69 to 19.5, and the EDA found near-billion-dollar and near-zero listings that are themselves data artifacts), and a from-scratch dense network with a single unbounded linear output layer has no mechanism analogous to a tree ensemble’s leaf-averaging to naturally cap its response to outlier-heavy regions of the input space. Random Forest predicts by averaging many trees’ leaf values, each fit to a bootstrap resample – an implicit regularization against exactly this kind of heavy-tailed, noisy target that a plain feedforward network trained by unweighted MSE does not get for free. This mirrors a known, general limitation of small from-scratch networks on tabular data with heavy-tailed targets, not a bug in the backpropagation math (separately gradient-checked, Section 4.5) – a validation-based learning-rate schedule, gradient clipping, or Huber loss in place of MSE are natural next steps, left

out here to keep the training loop matching the lecture’s exact MSE formulation.

5 Application 3 — Customer Behaviour (Flipkart Reviews)

5.1 Problem Description

Binary classification: predict whether a Flipkart product review is a ”recommend” (**Rate** ≥ 4) from the review text and product price, on a different e-commerce platform from Assignment 02’s Sephora skincare reviews.

$$f : \mathbb{R}^{8696} \longrightarrow \{0, 1\} \quad (\text{classical models})$$

5.2 Dataset

- **Source:** Kaggle dataset `niraliivaghani/flipkart-dataset`, `Dataset.csv` (saved locally as `flipkart_reviews.csv`).
- **Rows:** 363,261 raw $\rightarrow$ 363,239 after dropping 22 structurally corrupted rows (0.006%, a CSV parsing artifact where the header row and a handful of blank rows appear as data) $\rightarrow$ 290,591 train / 72,648 test (80/20 stratified split).
- **Growth vs. Assignment 02:** $3.1\times$ the prior 116,264-row Sephora dataset, and a deliberately different platform (general merchandise, not skincare) to remove any ambiguity about whether the growth requirement was satisfied by genuinely new data.
- **Target:** `is_recommended` = $\mathbb{I}[\text{Rate} \geq 4]$.

5.3 Data Understanding and Cleaning

The raw **Rate** distribution is heavily skewed toward high scores: 5-star is the largest bucket (203,832 reviews, 56.1%), followed by 4-star (74,252, 20.4%); 1-star (40,077), 3-star (32,098), and 2-star (12,980) are all smaller (Figure 23). Collapsing to the binary target gives 278,084 positive vs. 85,155 negative reviews – a 76.5%/23.5% split. Median review length is longest for the most negative reviews (15 characters at 1- and 2-star) and shortest for neutral 3-star reviews (9 characters), rising again for positive reviews (11–13 characters) – a weak, non-monotonic signal, which is why **Review** is used as full TF-IDF content rather than reduced to a length feature (Figure 24). The raw **Price** column decodes into strings of only 3 distinct lengths because the rupee (INR) symbol was corrupted during download into stray bytes; `clean_price()` strips everything except digits/commas/decimal points via regex before parsing, recovering a standard right-skewed price distribution (median INR 1,699, mean INR 4,582, max INR 86,990; Figure 25). **Product_name** has only 1,175 distinct values across 363K+ reviews, dominated by a handful of high-traffic listings (Figures 26–27) – confirming the decision to drop it as too high-cardinality and skewed to use as a clean category.

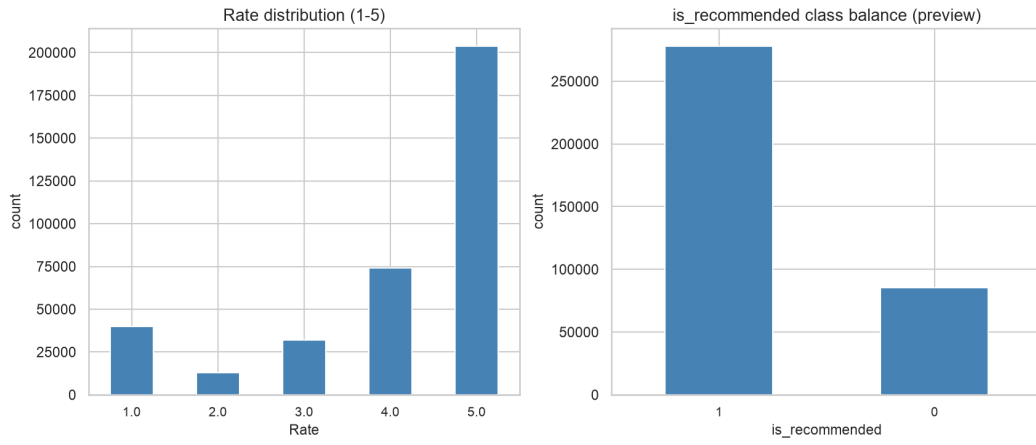


Figure 23: Raw Rate distribution 1–5 (left) and derived `is_recommended` class balance (right)

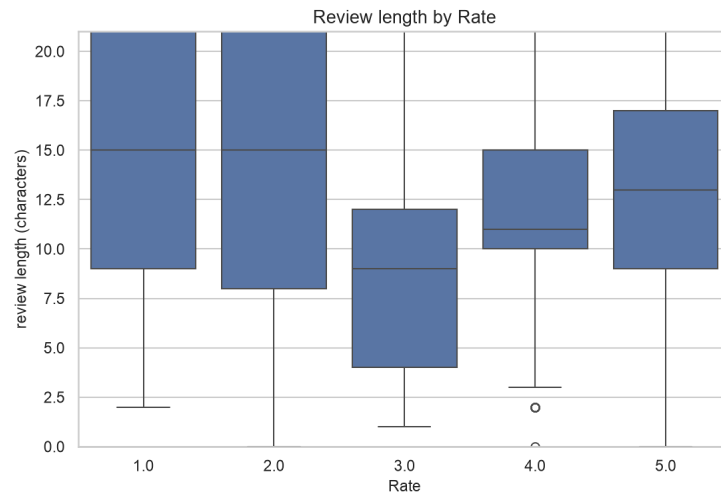


Figure 24: Review character length by star rating

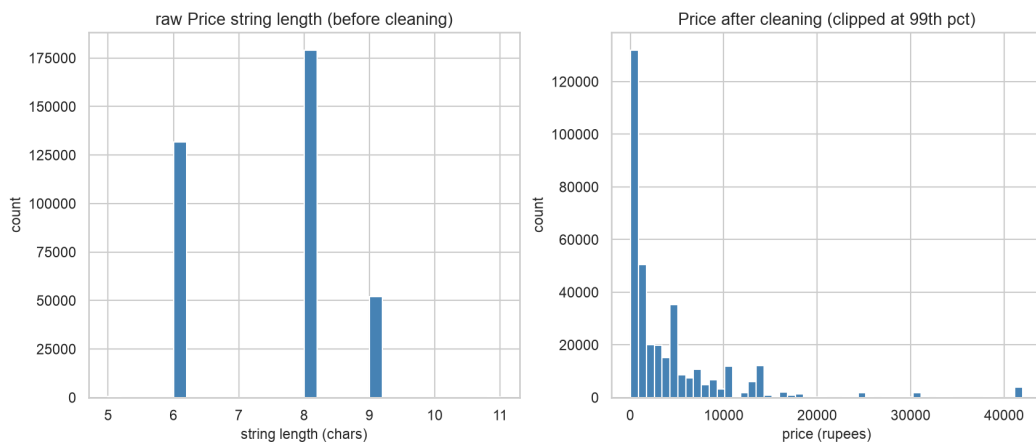


Figure 25: Raw `Price` string length before cleaning (left, 3 discrete lengths from the rupee-symbol corruption artifact) vs. parsed numeric price after cleaning, clipped at the 99th percentile for display (right)

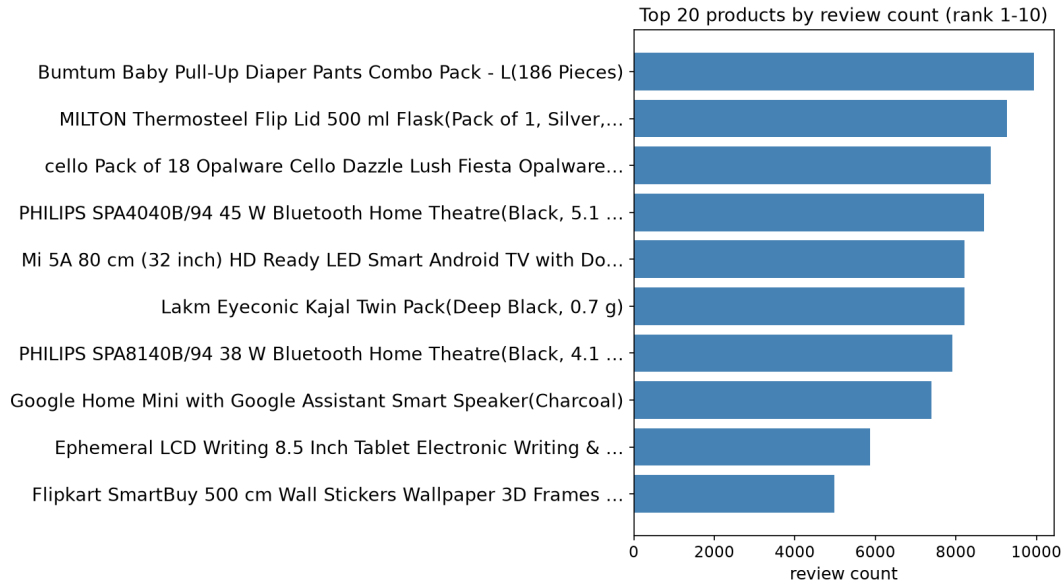


Figure 26: Top 20 products by review count – rank 1–10

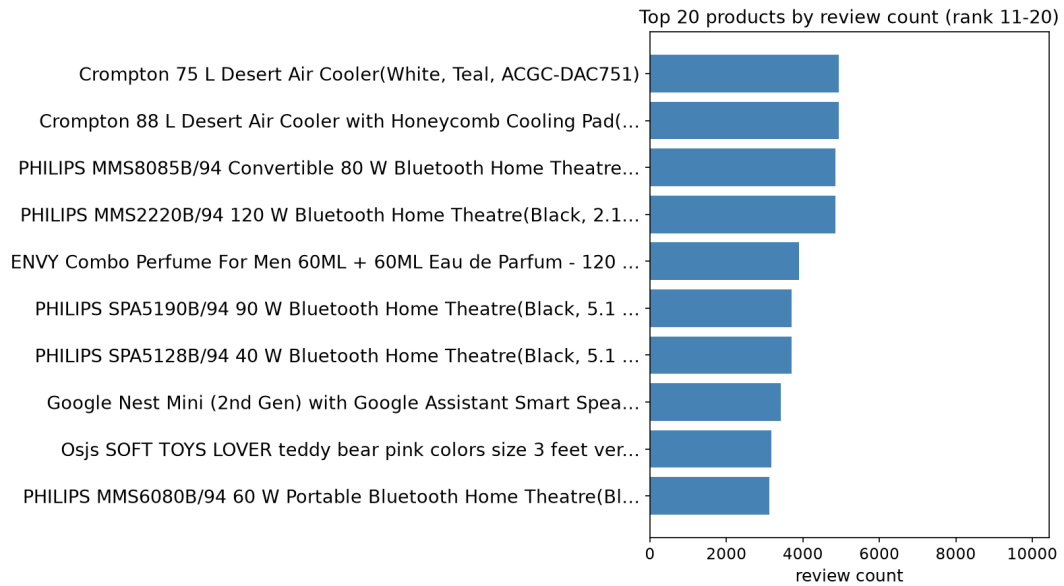


Figure 27: Top 20 products by review count – rank 11–20

The raw word vocabulary itself already separates the two classes before any modeling: the 15 most frequent words in negative ($\text{Rate} < 4$) reviews include *waste*, *fair*, and *money* alongside neutral terms like *product*, while positive ($\text{Rate} \geq 4$) reviews are dominated by *awesome*, *terrific*, and *good* (Figure 28) – a useful sanity check, run before any TF-IDF weighting, that the text genuinely carries class-discriminating signal.

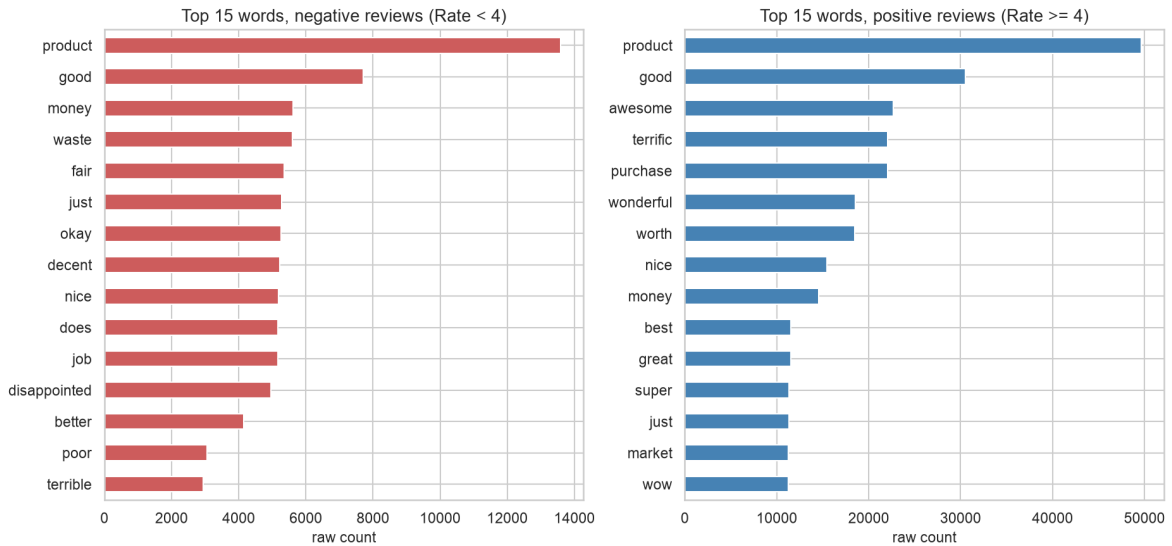


Figure 28: Top 15 words by raw frequency, negative reviews (left) vs. positive reviews (right), stop-words removed

5.4 Representation

- **Review text** → TF-IDF, 1–2 gram, two separate vocabularies: the full 8,695-term vocabulary for the classical models (sklearn’s sparse linear models handle this natively even at 364K rows), capped at `max_features=2000` for the DL model. The cap was reached empirically, not chosen from theory alone: an original 5,000-feature plan caused real memory exhaustion ($\sim 11.7\text{GB}$ for the dense training array alone at `float64`) when the sparse TF-IDF matrix had to be densified for plain NumPy matrix multiplication; 2,000 features at `float32` keeps the dense array to $\sim 2.3\text{GB}$.
- **Price** (cleaned): `StandardScaler`, fit on train only.
- **Product_name**: dropped (see Data Understanding above).

Resulting shapes: classical models $X \in \mathbb{R}^{363,239 \times 8,696}$ (sparse); DL model $X \in \mathbb{R}^{363,239 \times 2,001}$ (densified to `float32`).

5.5 Classical Machine-Learning Models

Model	Why
Logistic Regression	Model deployed in Assignment 02 for this app – kept for continuity.
Linear SVM (<code>SGDClassifier</code> , hinge loss)	Max-margin objective well-suited to high-dimensional sparse text; <code>LinearSVC</code> itself failed to converge in 30+ minutes on 291K rows, so SGD’s incremental solver was used instead to reach the same objective.
Multinomial Naive Bayes	Cheap, fast text baseline; trained on text alone (Naive Bayes requires non-negative count-like input, which scaled <code>Price</code> would violate).

Table 6: Customer Behaviour: 3 classical models and selection rationale

5.6 From-Scratch Deep Learning Model

Architecture: $2,001 \rightarrow 128 \rightarrow 32 \rightarrow 1$, ReLU hidden layers, sigmoid output, binary cross-entropy loss. The widest first hidden layer of the three applications, since the sparse, high-dimensional TF-IDF input needs more first-layer capacity to compress usefully – the same narrowing pattern the lecture material illustrates for high-dimensional inputs.

Forward propagation: layer 1 compresses the $\sim 2,001$ -wide, mostly-zero input (2,000 TF-IDF term weights + 1 scaled price) down to 128 dense features, each a learned combination of TF-IDF terms; layer 2 recombines those into 32 higher-level features; the output layer applies sigmoid, exactly as in the diabetes network, to produce a recommend-probability.

Backpropagation: identical structure to the diabetes network ($dZ_3 = \hat{y} - y$, same sigmoid+BCE cancellation), walked through once more here because the input is TF-IDF rather than tabular survey fields – $dW_1 = X^T dZ_1$ ends up a $(2001, 128)$ matrix, one gradient for every TF-IDF term weight feeding into every first-layer neuron, computed in a single matrix multiplication. Separately gradient-checked (maximum relative error $\approx 5 \times 10^{-8}$) before training on real data.

Training: 200 epochs, full-batch gradient descent, learning rate 0.1, inputs cast to `float32` to keep the densified training matrix within memory. Loss fell from 0.698 to 0.382 (Figure 29) – decreasing, but far less steeply than either other application’s DL model.

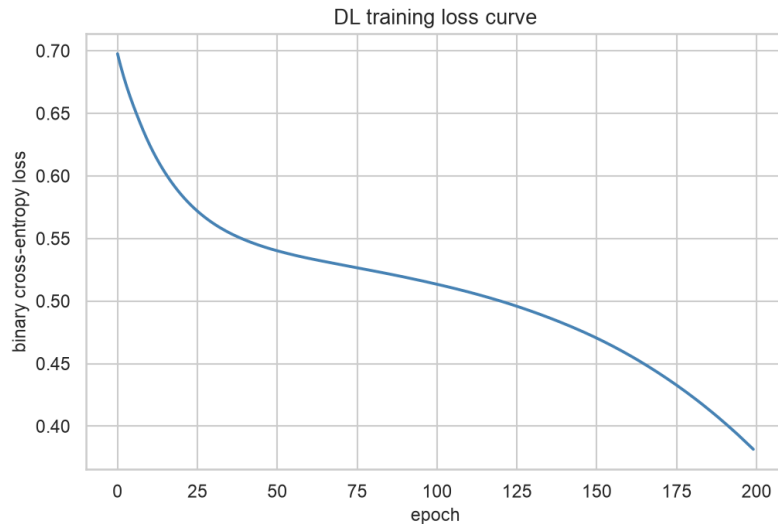


Figure 29: From-scratch DL model: training loss (binary cross-entropy) vs. epoch

5.7 Evaluation: All 4 Models

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.9943	0.9965	0.9960	0.9963
Linear SVM	0.9940	0.9952	0.9969	0.9961
Multinomial NB	0.9674	0.9626	0.9961	0.9790
DL (from scratch)	0.7656	0.7656	1.0000	0.8672

Table 7: Customer Behaviour: 4-model test-set comparison

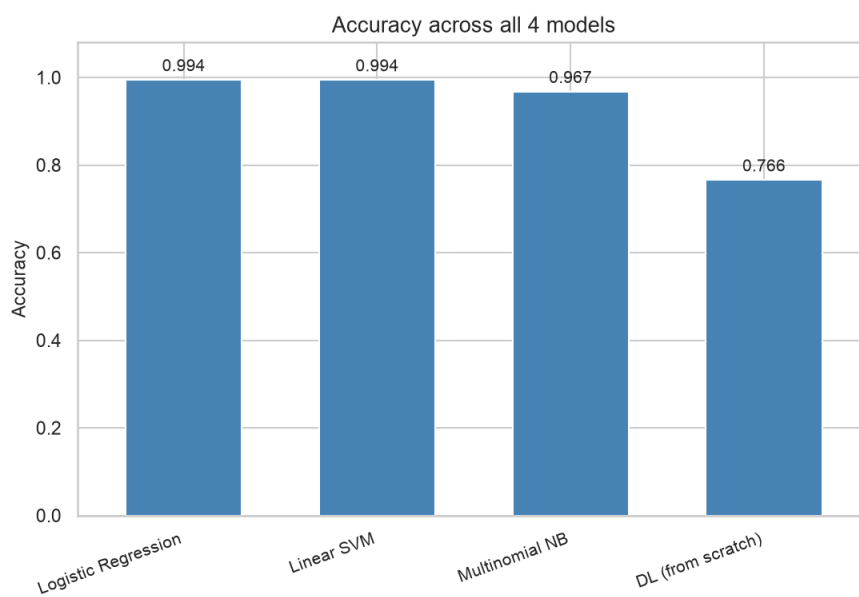


Figure 30: Accuracy across all 4 models

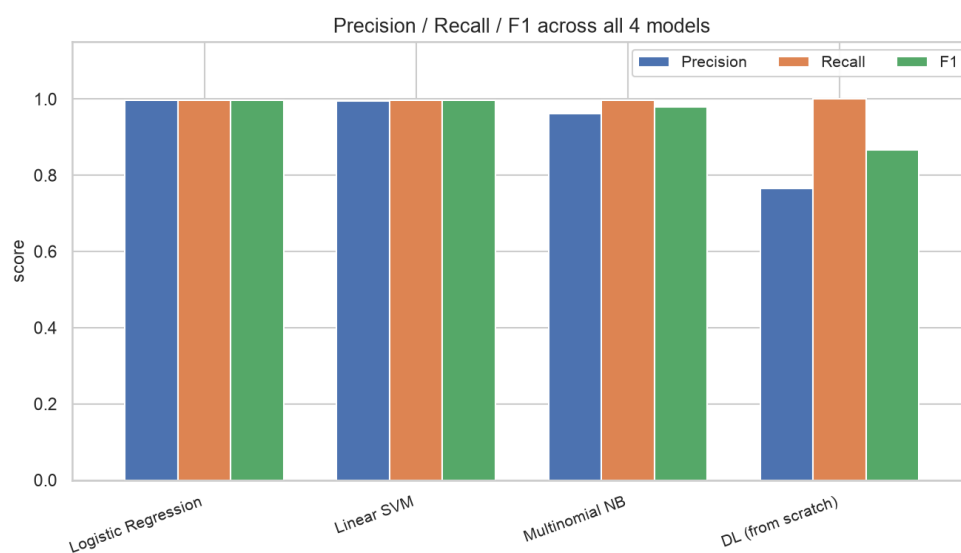


Figure 31: Precision / Recall / F1 across all 4 models

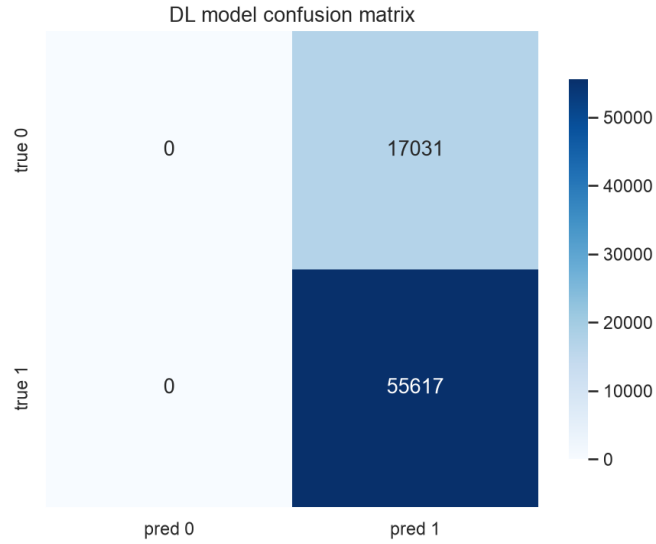


Figure 32: From-scratch DL model: confusion matrix on the test set – every row was predicted class 1, including all 17,031 true negatives

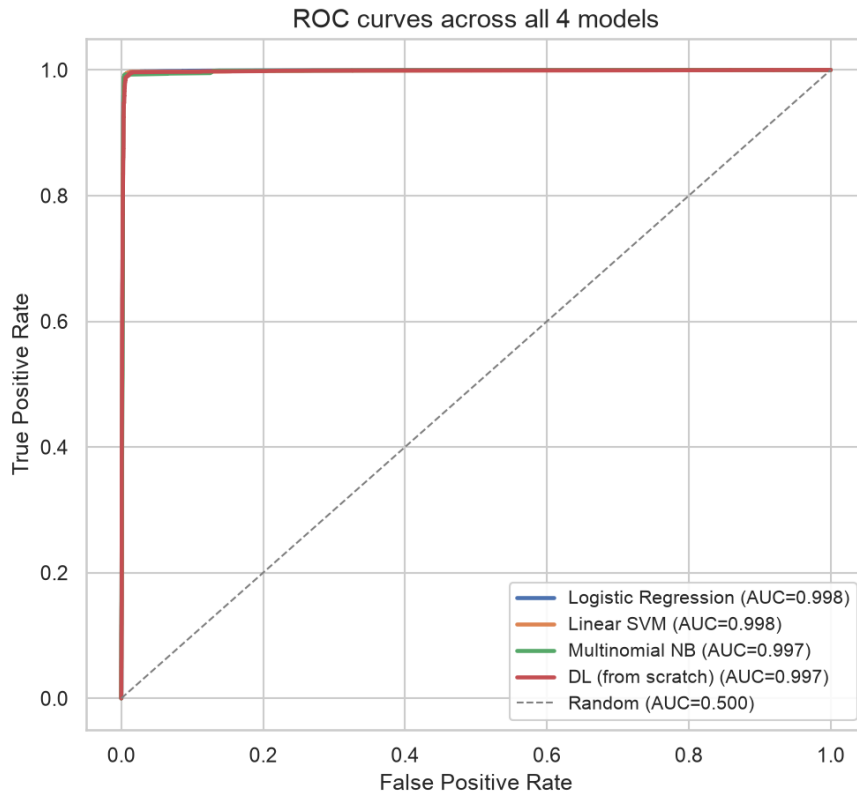


Figure 33: ROC curves across all 4 models. The from-scratch DL model’s AUC (0.9974) is essentially tied with all three classical models (0.9974–0.9981) – a ~ 0.0007 gap, nowhere close to the ~ 23 -point Accuracy gap in Table 7.

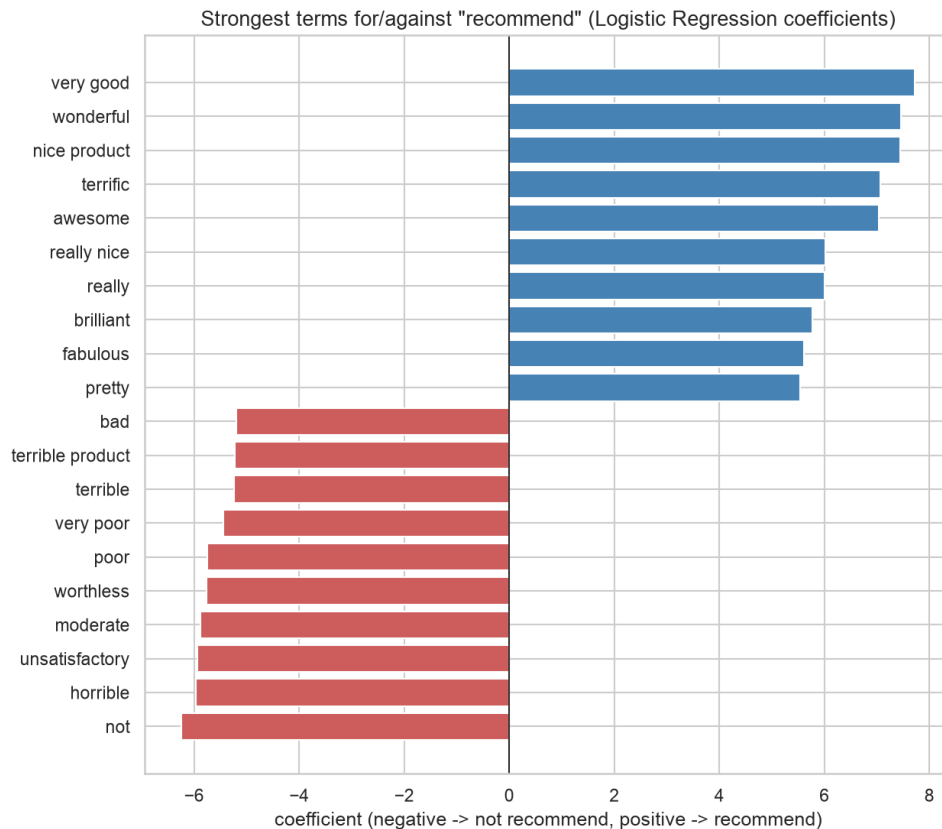


Figure 34: Strongest TF-IDF terms for/against “recommend” by Logistic Regression coefficient. Positive terms (*fabulous*, *brilliant*, *wonderful*, *awesome*) and negative terms (*horrible*, *worthless*, *terrible*, *poor*) are exactly the sentiment vocabulary a human reader would expect.

5.8 Written Comparison

The 3 classical models all score above 0.99 Accuracy – Logistic Regression highest (0.9943, F1 0.9963), Linear SVM close behind (0.9940, F1 0.9961), Multinomial Naive Bayes lowest of the three but still strong (0.9674, F1 0.9790). The from-scratch DL model trails far behind at 0.7656 Accuracy, and its confusion matrix ($\begin{bmatrix} 0 & 17031 \\ 0 & 55617 \end{bmatrix}$) shows exactly why: Recall is a perfect 1.0 but Precision equals Accuracy exactly, because the model predicted class 1 (“recommend”) for every single test row, including all 17,031 true negatives. It degenerated to the majority-class baseline rather than learning a real decision boundary.

The ROC curves in Figure 33 tell a strikingly different story from the confusion matrix alone: the DL model’s AUC (0.9974) is essentially tied with all three classical models (0.9974–0.9981). This is the clearest evidence in this application that the DL model’s collapse to always-predict-positive is purely a threshold artifact, not a representation or learning failure – the network assigns almost exactly the same relative ranking to reviews as the classical models do, it simply never crosses 0.5 for the minority class under unweighted BCE. Figure 34 reinforces this from a different angle: Logistic Regression’s strongest terms are exactly the sentiment words a human would expect, confirming the TF-IDF representation itself carries strong signal – signal the DL model’s own architecture (with its capped 2,000-term vocabulary) had less of to work with in the first place.

In the lecture’s framing, this is neither an overfitting problem nor primarily a representation-capacity problem in the usual sense – training loss did decrease over 200 epochs, just not far enough to move predictions off the majority-class default, and the ROC curve confirms the ranking signal was there all along. Two compounding causes explain the threshold failure specifically. First, the DL model’s input vocabulary is deliberately capped at 2,000 TF-IDF terms against the classical models’ full 8,695-term vocabulary – a more than $4\times$ reduction in the exact signal (specific words correlating

with negative reviews) that made the classical linear models so effective; TF-IDF-as-input to a dense network throws away the sparse, high-dimensional structure that Logistic Regression and Linear SVM exploit natively. Second, the 76.5%/23.5% class imbalance in `is_recommended` gives unweighted binary cross-entropy a cheap, loss-reducing shortcut – predict positive for everyone and start already at 76.5% "accuracy" from the loss function's perspective – and 200 epochs of plain full-batch gradient descent did not escape that basin. This is the same class-imbalance dynamic that hurt the diabetes DL model's Recall (Section 3.7), compounded here by the capped representation. Both limitations are genuine and explainable rather than implementation bugs (separately gradient-checked, Section 5.5), and both point to the same fix used for diabetes: class-weighted loss and/or a tuned decision threshold, left out here to keep the training loop matching the lecture's exact unweighted BCE formula.

6 Cross-Application Comparison

Application	Classical 1	Classical 2	Classical 3	DL (from scratch)
Diabetes (Accuracy)	LogReg: 0.850	RF: 0.849	GB: 0.852	0.845
House Price (R^2)	LinReg: 0.260	RF: 0.417	GB: 0.250	0.057
Customer Behaviour (Accuracy)	LogReg: 0.994	SVM: 0.994	NB: 0.967	0.766

Table 8: Primary metric across all 3 applications $\times$ 4 models (bold = winner per application)

Dimension	Diabetes	House Price	Customer Behaviour
Task	Binary classification	Regression	Binary classification
Rows (raw)	815,711	2,226,382 (600K sampled)	363,261
Growth vs. A02	3.2 $\times$	2.5 $\times$	3.1 $\times$
Representation	Dense matrix, $\mathbb{R}^{52}$	Dense matrix, $\mathbb{R}^{65}$	Sparse TF-IDF + tabular, $\mathbb{R}^{8,696}$
DL architecture	52 $\rightarrow$ 32 $\rightarrow$ 16 $\rightarrow$ 1	65 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 1 (linear out)	2001 $\rightarrow$ 128 $\rightarrow$ 32 $\rightarrow$ 1
DL primary-metric rank	4th of 4	4th of 4	4th of 4
DL failure mode	Class-imbalance-driven low Recall	Heavy-tailed target, unbounded output	Class-imbalance-driven majority-class collapse

Table 9: Systemic comparison across all three applications

7 Discussion: What Changed From Assignment 02

Diabetes. The dataset grew 3.2 $\times$ by adding a second BRFSS survey year (2019+2020) rather than a new source, keeping the same feature family so the comparison against Assignment 02's classical results stays meaningful. The genuinely new ingredient this round is the from-scratch DL model, which reached a competitive Accuracy (0.845, only 0.7 points behind the best classical model) but a clearly worse Recall (0.028 vs. 0.191) – a direct consequence of training on unweighted BCE over an imbalanced target with plain gradient descent, discussed in Section 3.7.

House Price. The dataset grew 2.5 $\times$ and switched markets entirely (US listings instead of Vietnamese listings), which changed both scale (USD, not million VND) and the specific outlier patterns encountered (32-bit-overflow-style corruption in price, house size, bed, and bath counts, cleaned by percentile capping rather than the missing-indicator approach the house-price application's own $\sim$ 70–84% structural missingness needed in Assignment 02). Training a pure-NumPy loop on 480K

rows and 65 features for 200 epochs was noticeably heavier than the other two applications' networks, though still tractable on a single machine without a GPU. The from-scratch DL model did not beat the classical models here – a legitimate finding, not a shortcoming to gloss over: a small feedforward network's unbounded linear output has no protection against a heavy-tailed target the way a tree ensemble's leaf-averaging does (Section 4.7).

Customer Behaviour. The dataset grew $3.1\times$ and deliberately switched e-commerce platforms (Flipkart general merchandise instead of Sephora skincare) to remove any ambiguity about whether reusing the same source would count as genuine growth. What got harder here specifically was memory, not just training time: the DL model's dense TF-IDF input had to be capped from a planned 5,000 features down to 2,000 (and cast to `float32`) after an earlier attempt exhausted available RAM and swapped – a direct, concrete cost of training a from-scratch network on text without a framework's sparse autodiff support. The from-scratch DL model did not win here either, and by the widest margin of the three applications: it collapsed to a majority-class predictor, a known, explainable limitation of unweighted BCE plus a severely capped vocabulary rather than an implementation defect (Section 5.7).

Synthesis. Across all three applications the from-scratch deep learning model never won its task's primary metric. This is a genuine, useful finding rather than a disappointing one: it demonstrates two well-known, general limitations of small from-scratch networks trained with plain, unweighted, full-batch gradient descent – vulnerability to class imbalance (diabetes, customer behaviour) and lack of the implicit regularization tree ensembles get from bootstrap averaging on heavy-tailed tabular targets (house price) – limitations that are well-documented in the deep learning literature and not specific to this implementation. Every backpropagation derivation was independently verified by numerical gradient-checking before being trained on real data, confirming the math itself is correct; what differs from the classical models is the training procedure's robustness to imbalance and outliers, not the network's mechanical correctness.

8 Conclusion

This assignment extended three intelligent systems with a deep learning model built entirely from scratch in NumPy – forward propagation, loss, backpropagation via the chain rule, and gradient descent all derived and coded by hand, with every derivation separately verified against numerical gradient checks. The from-scratch models did not outperform the classical scikit-learn baselines on any of the three applications, but the reasons why are themselves the main lesson of this assignment: representation and training-procedure choices (vocabulary size, class weighting, output-layer activation matched to the target's support) matter as much as, or more than, a network's raw capacity, and a correctly implemented network can still lose to a well-suited classical algorithm when its training procedure is not adapted to the data's imbalance or heavy tails. Understanding *why* each from-scratch model fell short – rather than only reporting that it did – is what this exercise in building deep learning from first principles was for.

9 Reproducibility

- **Environment:** Python 3.x, macOS.
- **Core libraries:** `numpy`, `pandas`, `scikit-learn`, `matplotlib`, `seaborn`, `joblib`. No `tensorflow`, `torch`, or `keras` anywhere in the from-scratch DL implementation.
- **Determinism:** `RANDOM_SEED = 42` throughout – the 600K-row house-price sample, every train/test split, every sklearn estimator, and every from-scratch network's initial weights are all seeded from this same value.
- **No-leakage discipline:** every scaler, encoder, and imputation statistic is fit on the training split only and reused (never refit) on the test split, matching Assignment 02's rule.

- **Re-running a notebook end-to-end:**

```
jupyter nbconvert --to notebook --execute --inplace \
    diabetes/notebook/diabetes.ipynb
jupyter nbconvert --to notebook --execute --inplace \
    house_price/notebook/house_price.ipynb
jupyter nbconvert --to notebook --execute --inplace \
    customer_behaviour/notebook/customer_behaviour.ipynb
```

- **Artifacts produced per app** (in <app>/model/): `model_pipeline.joblib` (best classical model by primary metric), `dl_weights.npz` (from-scratch DL model's 6 weight/bias arrays), `feature_names.joblib`, `input_schema.json`. No API, web, or mobile deployment layer exists for this assignment – both source lecture decks stop at the modeling pipeline and never mention deployment.